

GRADO

SERIE GUÍAS · TECNOLOGÍA E INFORMÁTICA



8°

Lógica avanzada I: estructuras condicionales anidadas (if-else-if) y operadores lógicos AND/OR/NOT

Guía 11-8-TIC

Periodo Segundo

Institución Educativa Sor María Juliana

Autor: Dr. Álvaro Cárdenas Orozco

Metodología MILC · Apertura ancestral · Pensamiento computacional · Triángulo Dussel-Estoicismo-Florida

Producto: Diagrama de flujo + bloque MakeCode con 3 condiciones anidadas + tabla de verdad de 1 expresión compuesta verificada.

Apertura: Lógica avanzada I: estructuras condicionales anidadas (if-else-if) y operadores lógicos AND/OR/NOT

Saber ancestral

Las **pruebas del oficio del herrero** en los talleres tradicionales: ¿el hierro está caliente Y bien limpio Y el martillo a la altura correcta? Si falla cualquiera, la pieza se rompe. La decisión depende de varias condiciones que se cruzan al mismo tiempo. Las condiciones lógicas son oficio antes de ser código.

Saber contemporáneo

Las **estructuras condicionales** en programación (if-else-if) y los operadores lógicos (AND, OR, NOT) traducen al código las decisiones del oficio: “*si esto Y aquello, entonces actúa*”. Son la columna vertebral de cualquier programa que toma decisiones.

Pregunta-puente

¿Qué del juicio del herrero —su capacidad de cruzar varias condiciones antes de actuar— podemos llevar al código que decide?

Saber y saber hacer

Al terminar podrás **escribir condicionales anidadas** (if-else-if), **combinar** con operadores lógicos AND/OR/NOT, **construir tablas de verdad** para verificar tu lógica y **depurar** cuando una condición no se cumple como esperabas.

Autor	Dr. Álvaro Cárdenas Orozco
DBA	Aplica estructuras condicionales y operadores lógicos para resolver problemas que requieren múltiples decisiones simultáneas.
Referentes	MEN Tecnología · ICFES Pensamiento Computacional · Modelo MILC · CSTA Standards · Floridi (decisiones informacionales) · Estoicismo (lo que depende de mí)
Producto final	Diagrama de flujo + bloque MakeCode con 3 condiciones anidadas + tabla de verdad de 1 expresión compuesta verificada.

Ruta MILC: así vamos a aprender

1

Escuta
Observo, escucho
y pregunto.

2

Sistematización
Pensamiento
computacional.

3

Praxis
Construyo y aplico.

4

Evaluación
Reflexión liberadora.

Fase 1 · Escuta

Escena para escuchar

Felipe modela en MakeCode el semáforo del cruce frente al colegio. La regla es: Si hay peatón Y la luz está en verde para autos, los autos paran. Pero se da cuenta que falta otra condición: ¿qué pasa si hay peatón pero está en la acera, no cruzando? La condición simple no alcanza. Necesita anidar y combinar lógica.

- ☐ Recuerdo una decisión real donde tuve que cruzar varias condiciones a la vez.
- ☐ Identifico la diferencia entre AND (Y) y OR (O) en español cotidiano.
- ☐ Distingo cuándo conviene anidar condicionales (if-else-if) en lugar de varias separadas.

Una decisión de mi vida con varias condiciones a la vez fue:

Las condiciones que tuve que cruzar fueron:

Si la programara, mi pseudocódigo empezaría con:

Saber más. Los operadores lógicos vienen de George Boole (s. XIX). Su "álgebra booleana" es la base de TODA la computación moderna: cada decisión que toma tu celular se reduce a combinaciones de AND, OR, NOT. Lo que el herrero hacía con intuición, hoy se calcula con lógica formal.

Fase 2 · Sistematización con pensamiento computacional

Cuatro pilares aplicados al tema

El pensamiento computacional descompone la decisión en sus condiciones, reconoce patrones de combinación lógica, abstrae la regla *toda decisión es función de variables* y diseña un algoritmo verificable con tabla de verdad.

Pilar	Aplicación al tema
1. Descomposición	Descompón la decisión: ¿cuántas condiciones hay? ¿se necesitan todas (AND) o basta una (OR)? ¿hay condiciones que invierten (NOT)?
2. Reconocimiento de patrones	Patrones: AND para cruce obligatorio (todas verdaderas). OR para alternativa (al menos una). NOT para inversión.
3. Abstracción	Abstracción: la condición compuesta se puede escribir como UNA expresión booleana que evalúa V o F.
4. Algoritmo conceptual	Algoritmo: 1) listar condiciones · 2) elegir operadores · 3) construir expresión · 4) crear tabla de verdad · 5) verificar todos los casos · 6) traducir a código.

Anatomía de una expresión booleana

Variables (proposiciones): cada condición que evalúa V/F. Ej: hay_peaton, luz_verde.

Operador AND (Y): ambas verdaderas. Ej: hay_peaton AND luz_verde.

Operador OR (O): al menos una verdadera. Ej: lluvia OR frio.

Operador NOT (no): invierte. Ej: NOT hay_peaton.

Paréntesis: agrupan y fuerzan orden. Ej: (A OR B) AND C.

Tabla de verdad: todos los casos posibles para verificar la expresión.

Errores comunes y su corrección

- Confundir AND con OR en lenguaje natural → “si llueve y hace frío” (AND) vs “si llueve o hace frío” (OR) son DOS reglas distintas.
- Olvidar paréntesis → A OR B AND C se evalúa diferente a (A OR B) AND C.
- No probar todos los casos → una tabla de verdad de 3 variables tiene 8 filas; verifícalas todas.
- Anidar demasiado → if dentro de if dentro de if se vuelve ilegible; usa AND/OR cuando puedas.
- Negar mal → NOT (A AND B) es NOT A OR NOT B, no NOT A AND NOT B.

Mi expresión booleana del semáforo del cruce sería:

Construida la tabla de verdad, los casos verdaderos son:

Fase 3 · Praxis con pensamiento computacional

Construyendo el producto con los cuatro pilares

Construir un sistema de decisión con condicionales anidadas es un sistema. Decomponemos las condiciones, reconocemos patrones lógicos, abstraemos la expresión y seguimos un algoritmo verificable.

Pilar	Aplicación al producto
Descomposición	Sistema: 1) variables · 2) condiciones · 3) operadores · 4) anidamiento · 5) tabla de verdad · 6) código.
Patrones	Patrones de buenas decisiones programadas: condición clara, anidamiento mínimo, tabla de verdad documentada.
Abstracción	Función esencial: el código toma UNA decisión correctamente para todos los casos posibles.
Algoritmo de armado	Pasos: definir variables → escribir condiciones → combinar con operadores → verificar con tabla → traducir a MakeCode.

Checklist del sistema de decisión

- ☐ 3 variables booleanas identificadas claramente.
- ☐ Expresión compuesta usando AND, OR y NOT.
- ☐ Tabla de verdad completa (2^n filas).
- ☐ Diagrama de flujo del algoritmo.
- ☐ Bloque MakeCode con la lógica.
- ☐ Prueba con al menos 3 casos distintos.

Plantilla de trabajo

Variables: A = __, B = __, C = __.

Expresión: (A AND B) OR (NOT C) (ejemplo).

Tabla de verdad: 8 filas ($2^3 = 8$ casos posibles).

MakeCode: if (A and B) or (not C) then acción.

Prueba: caso 1 (A=V, B=V, C=F) → V. caso 2 (A=F, B=F, C=V) → F.

Documentación: explicar qué representa cada variable en el contexto real.

Mi sistema de decisión: variables + expresión + tabla + código MakeCode:

Producto de la sesión

Diagrama de flujo + bloque MakeCode + tabla de verdad de expresión compuesta.

Criterios de calidad

- 3 variables booleanas con significado real.
- Expresión usando AND, OR, NOT con paréntesis correctos.
- Tabla de verdad completa y verificada.
- Diagrama de flujo legible.
- Bloque MakeCode funcional.
- Documentación de qué hace cada parte.

Fase 4 · Evaluación liberadora — cinco dimensiones

Sentido de la evaluación

Evaluar no es solo revisar una nota. Es reconocer qué aprendí, qué cambió en mí, qué emociones manejé, cómo me relaciono con la ciudad y la comunidad, y qué le dejaré a quien viene detrás.

Mi reflexión liberadora en cinco dimensiones. Completa cada frase iniciada:

Dimensión	Mi respuesta (frame starter)	Algo que descubrí
1. Personal	Lo que más cambió en mí fue _____	_____
2. Emocional	La emoción más fuerte fue _____ y la regulé _____	_____
3. Ciudadana	En democracia esto importa porque _____	_____
4. Local (Cartago)	En mi barrio sería útil para _____	_____
5. Intergeneracional	A mi abuela/abuelo le diría _____	_____

Para la próxima sustentación. Vuelve a esta tabla en seis meses. Verás que las respuestas cambian — no porque lo aprendido cambie, sino porque tú cambias. La evaluación liberadora es una conversación contigo a través del tiempo.

Triángulo de pensamiento · cierre filosófico

Dussel · lente del nosotros

“La analéctica supera a la dialéctica al partir del Otro como Otro.”

Toda regla lógica excluye casos que no contempla. Una condición `if mayor_de_edad` excluye al adolescente que necesita una decisión urgente. La analéctica con código pregunta: ¿qué caso real quedó fuera de mi expresión booleana?

¿Qué caso real (persona, situación) NO captura tu expresión condicional? ¿Cómo lo incluirías?

Estoicismo · Marco Aurelio · cuidado interior

“El obstáculo en el camino se vuelve el camino.”

Cada error de lógica (un AND donde debías poner OR) es maestro: la tabla de verdad te muestra dónde fallaste y por qué. La depuración es ejercicio estoico de aprender del error.

¿Qué error de lógica reciente se convirtió en aprendizaje permanente?

Floridi · lente de la infoesfera

“Cada decisión de información tiene peso ético.”

Una condición mal escrita en un software de salud (ej: `if dosis >100` en lugar de `if dosis <100`) puede dañar a alguien real. La lógica no es ejercicio académico — es responsabilidad sobre vidas.

Si tu sistema de decisión se aplicara a una situación real (semáforo, alarma médica), ¿quién se afectaría si fallas la condición?

Compromiso final

Criterio	Lo logré	En proceso	Necesito apoyo
Comprensión del tema	Explico ideas centrales con mis palabras.	Reconozco ideas pero falta precisión.	Necesito repasar conceptos base.
Pensamiento computacional	Apliqué los 4 pilares al diseñar mi producto.	Apliqué algunos pilares.	Necesito releer la fase 2.
Aplicación y producto	Mi producto cumple los criterios y se puede revisar.	Producto funcional con ajustes pendientes.	Aún en construcción.
Reflexión 5 dimensiones	Respondí cada dimensión con honestidad.	Respondí algunas con detalle.	Mis respuestas son muy generales.
Triángulo de pensamiento	Conecté las 3 voces con mi trabajo real.	Conecté al menos una.	No relaciono las voces con mi proceso.

Hoy entendí que:

Mi compromiso para la próxima sesión es:

Cita de despedida · Marco Aurelio

“El obstáculo en el camino se vuelve el camino. Lo que estorba al camino se vuelve camino.”
Cada error de hoy, bien observado, se convierte en pieza del camino que pisarás mañana.

Nombre completo:

Fecha: _____ **Curso/grupo:** _____

Mi firma: _____

Próxima sesión. Profundizaremos en tablas de verdad y condiciones compuestas. La lógica del herrero se vuelve disciplina booleana.